



Guanaco Advisor

Asistente de IA local · ADEN

Sistema de preguntas y respuestas (RAG) 100 % local sobre documentos propios e información del ERP Odoo, con modelos de inteligencia artificial ejecutados en infraestructura propia.

1. Resumen ejecutivo

Guanaco Advisor es un sistema de **RAG** (*Retrieval-Augmented Generation*) que permite hacer preguntas en lenguaje natural y obtener respuestas basadas en documentos propios (PDF, TXT, DOCX) y en datos del ERP **Odoo**.

La clave del proyecto es que **todo corre localmente**: los modelos de lenguaje y de embeddings se ejecutan con **Ollama** en el propio servidor, y la base de conocimiento vive en una base vectorial local (**Chroma**). No hay dependencia de APIs externas ni se exponen datos sensibles a terceros.

Problema que resuelve: convierte información dispersa (documentos, ERP) en un asistente conversacional que responde con precisión y **cita las fuentes** que usó.

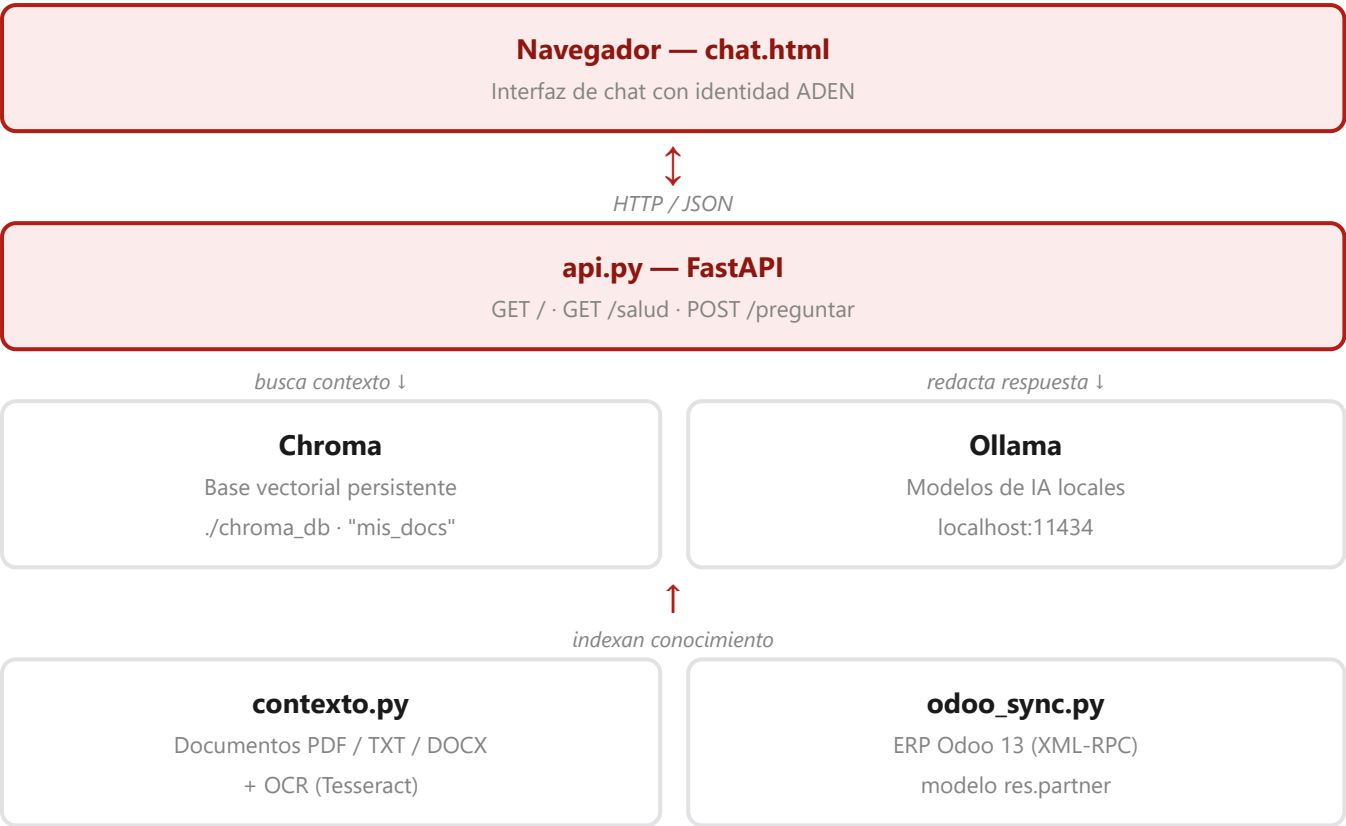
2. ¿Qué es RAG y por qué lo usamos?

Un modelo de lenguaje (LLM) por sí solo no conoce tus documentos ni tu base de datos. El patrón **RAG** lo resuelve en dos tiempos:

- 1 Recuperación:** ante una pregunta, se buscan los fragmentos de información más relevantes en la base vectorial.
- 2 Generación:** esos fragmentos se le pasan al LLM como contexto, y el modelo redacta la respuesta basándose en ellos.

Sin RAG	Con RAG (este proyecto)
Responde de memoria (puede "alucinar")	Responde con datos reales y verificables
No conoce tus documentos	Usa tus PDFs, DOCX y Odoo
No cita fuentes	Cita de qué documento/registro salió la respuesta
Reentrenar es caro	Agregar conocimiento = solo indexar más documentos

3. Arquitectura general



Componente	Tecnología	Rol
Interfaz	HTML/CSS/JS	Chat web con identidad ADEN
API	FastAPI + Uvicorn	Expone el RAG por HTTP
Orquestación RAG	LlamaIndex	Une búsqueda + LLM
Base vectorial	Chroma	Almacena los embeddings
Motor de IA	Ollama	Corre LLM y embeddings localmente
Ingesta documentos	contexto.py + Tesseract	Lee y vectoriza archivos
Ingesta Odoo	odoo_sync.py (XML-RPC)	Trae registros del ERP

4. Modelos de IA utilizados

Todos los modelos se ejecutan mediante **Ollama** en `localhost:11434`.

LLM · GENERACIÓN

gemma4:e2b

Redacta la respuesta final a partir del contexto recuperado.

EMBEDDINGS

nomic-embed-text

Convierte texto en vectores para la búsqueda semántica.

VISIÓN · OPCIONAL

llava

Describe imágenes en documentos (desactivado por defecto).

¿Qué es un embedding? Es la representación numérica (un vector de cientos de dimensiones) del *significado* de un texto. Textos con significado parecido quedan "cerca" en ese espacio, lo que permite buscar por similitud semántica en lugar de por palabras exactas.

Configuración centralizada: los nombres de los modelos están en `contexto.py` y son la única fuente de verdad: tanto la API como la sincronización de Odoos los reutilizan desde ahí.

5. Cómo funciona todo

5.1 · Indexación (carga de conocimiento) — offline

Ocurre al ejecutar `contexto.py` (documentos) u `odoo_sync.py` (Odoo).

- 1 **Leer fuente:** archivo (PDF/TXT/DOCX) o registro de Odoo.
- 2 **Extraer texto:** incluye OCR de imágenes si aplica.
- 3 **Trocear (chunking):** LlamaIndex divide el texto en fragmentos.
- 4 **Generar embeddings:** nomic-embed-text vectoriza cada fragmento.
- 5 **Guardar en Chroma:** vector + texto + metadatos (fuente, origen).

5.2 · Consulta (uso diario) — online

Ocurre en cada pregunta a través de la API.

- 1 El usuario escribe la pregunta en el chat.
- 2 `POST /preguntar` → la API recibe el texto.
- 3 La pregunta se convierte en embedding.
- 4 Chroma devuelve los **TOP_K (=4)** fragmentos más similares.
- 5 Esos fragmentos + la pregunta van al LLM (gemma4:e2b).
- 6 El LLM redacta la respuesta.
- 7 Se devuelve `{ respuesta, fuentes }` al navegador.

6. Documentación de la API

API HTTP construida con **FastAPI**, escucha en `http://0.0.0.0:8000`. El índice y los modelos se cargan **una sola vez** al arrancar; cada request solo hace la búsqueda y la generación.

GET /

Sirve la interfaz de chat (`chat.html`). Respuesta: `text/html`.

GET /salud

Estado del sistema y de la base vectorial.

```
{
  "estado": "ok",
  "modelo_llm": "gemma4:e2b",
  "modelo_embed": "nomic-embed-text",
  "vectores": 111000
}
```

POST /preguntar

Realiza una consulta RAG. **Body:**

```
{ "pregunta": "¿Qué contactos hay en Buenos Aires?" }
```

Respuesta:

```
{
  "respuesta": "Según los registros, los contactos en Buenos Aires son...",
  "fuentes": [
    { "fuente": "odoo:res.partner:123", "score": 0.8421 },
    { "fuente": "manual_inscripcion.pdf", "score": 0.7733 }
  ]
}
```

Campo	Tipo	Descripción
respuesta	string	Texto generado por el LLM
fuentes	array	Fragmentos usados, con su score de similitud (trazabilidad)

Importante: la API **no** reindexa. Para cargar conocimiento nuevo se corre `contexto.py` u `odoo_sync.py` y luego se **reinicia** la API.

7. Integración con Odoo

Módulo `odoo_sync.py` . Conecta a un **Odoo 13 self-hosted** vía **XML-RPC** (librería estándar `xmlrpc.client` , sin dependencias extra). Enfoque: **indexación** — los registros se traen, se convierten a texto y se indexan en la misma base Chroma que los documentos. Modelo sincronizado: `res.partner` (contactos).

- 1 Autenticación XML-RPC con Odoo (credenciales en `.env`).
- 2 Por cada modelo: borra los vectores previos (re-sync limpio).
- 3 Trae los registros paginados (tandas de 200).
- 4 Convierte cada registro a texto legible (resuelve relaciones a su nombre).
- 5 Indexa en Chroma con metadatos de trazabilidad.

Seguridad: las credenciales viven en un archivo `.env` que no se versiona (está en `.gitignore`). Se recomienda un usuario Odoo dedicado de **solo lectura**.

8. Stack tecnológico

Capa	Tecnología
Lenguaje	Python 3
Motor de IA local	Ollama
LLM	gemma4:e2b
Embeddings	nomic-embed-text
Orquestación RAG	LlamaIndex
Base vectorial	Chroma (persistente)
API web	FastAPI + Uvicorn
Frontend	HTML/CSS/JS (vanilla)
OCR	Tesseract
Integración ERP	Odoo 13 vía XML-RPC
Gestión de secretos	python-dotenv (.env)

9. Características destacadas

- ▶ **100 % local y privado** — ningún dato sale de la infraestructura propia.
- ▶ **Multi-fuente** — combina documentos (PDF/DOCX/TXT) y datos del ERP Odoo.
- ▶ **Trazabilidad** — cada respuesta cita las fuentes que la respaldan.
- ▶ **OCR integrado** — extrae texto incluso de imágenes dentro de documentos.
- ▶ **Indexación incremental** — agregar conocimiento no requiere reentrenar.
- ▶ **Interfaz con identidad ADEN** — lista para uso institucional.
- ▶ **Modular y extensible** — sumar fuentes nuevas es agregar un script de ingesta.

10. Trabajo futuro

- ▶ **Escala de indexación:** la carga inicial de grandes volúmenes (ej. ~111.000 registros) es intensiva; se puede acelerar con embeddings por lote.
- ▶ **Sync incremental de Odoo:** usar la fecha de modificación para traer solo lo que cambió.
- ▶ **Respuestas en streaming:** mostrar la respuesta token a token.
- ▶ **Control de acceso:** sumar autenticación si la API se expone fuera de la red local.